

university of groningen

Performance Engineering: Lecture #02

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-02

Overview



1. Performance Measurement
2. Compiler Chain
3. Assembler
4. Final

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Performance Measurement Methods



- Methods
  - **Profiling**: Characterize the behaviour of an application in terms of aggregate performance metrics
    - Example: Average time spent in a subroutine
  - **Sampling**: Creation of profiles by means of periodic or random probing of performance observables independently of the current state of application execution
  - **Event-tracing**: Generate a list of event records comprising time stamp, location, type information, event-specific information
- Typically, additional instructions need to be inserted into the application (**instrumentation**)

Typical Time Scales



- Assume a processor with $f = 2.5\,\text{GHz}$ clock frequency
- Typical figures:

Core clock cycle	$0.4\,\text{ns}$
L1 cache access latency	$3 - 4\,\text{clock cycles}$
L2 cache access latency	About $10\,\text{clock cycles}$
L3 cache access latency	$\geq 35\,\text{clock cycles}$
Memory access latency	$O(200 - 300)\,\text{clock cycles}$

☞ Need time resolution $\ll 1\,\mu\text{s}$

Time Measurements



- Different time-related metrics
  - **Elapsed real time** or **wall-clock time**
    - Time from start to end as it could be observed using an external timer
  - **Resource utilization time**
    - Time during which a task occupies a resource, e.g. a CPU core
    - Timer is on hold when resource is used for other purposes
  - **Cycle counter**
    - The counter increment rate depends on the clock frequency
    - There may be different clock domains with variable clock
      - Exception: Reference clock

Hardware Performance Counters



- Modern processors provide hardware counters organised in a **Performance Monitoring Unit (PMU)**
- **Hardware performance counters** = Small set of registers that count events
  - The counter is incremented whenever a particular event occurs
  - The PMU is reconfigurable
- **Caveat**: Events are not always well-defined

Hardware Performance Counters: Examples

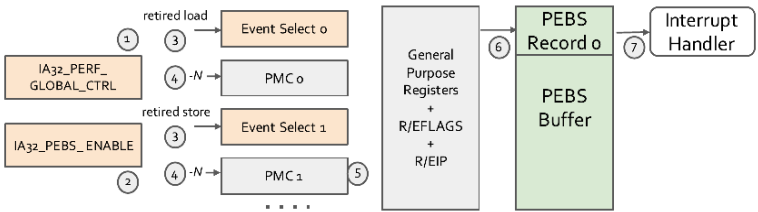


- Clock tick
- Instructions issued
- Integer instructions
- Load/store instructions
- ...
- Conditional branch instructions
- ...
- Level 1 data cache misses
- Level 1 instruction cache misses
- ...
- Translation lookaside buffer misses
- Hardware interrupts

Advanced Hardware Performance Counters

- Also known as: **precise event sampling**
- Examples:
  - Intel Processor Event Based Sampling (PEBS)
  - AMD Instruction-Based Sampling (IBS)
  - Arm Statistical Profiling Extension (SPE)

[Sasongko et al.; 2023]



Measurement Issues



- Issues related to performance measurement
  - Code instrumentation may impact performance
  - Accessibility of hardware counters
    - Performance counters by default require privileges
    - Support from the kernel may be required
  - Unclear or different definitions of hardware events
  - Management of large event traces

Deterministic versus Statistical Observables



- **Deterministic observables**:
  - Total number of committed instructions
  - Number of branch instructions
  - ...
- **Statistical observables**:
  - Execution time
  - Number of cache misses
  - Number of mispredicted conditional branch instructions
  - ...

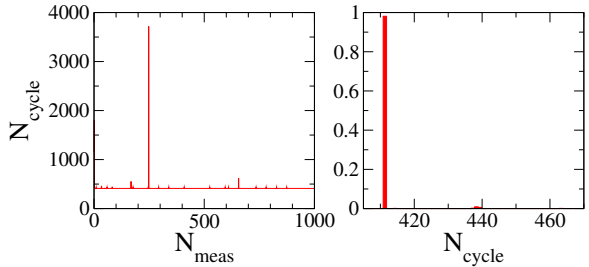
Variability



- Variability of the same system
  - Non-deterministic hardware (e.g. caches, memory management)
    - Example: Initially **caches are cold**
  - Non-uniform data paths (e.g. NUMA nodes)
  - Interference with other processes or (kernel-)threads (“OS jitter”)
  - Dynamic change of clock speed
- Variability between different but identical systems
  - Temperature variations
  - Manufacturing variability
  - “fail-slow” hardware

Variability: Real Example

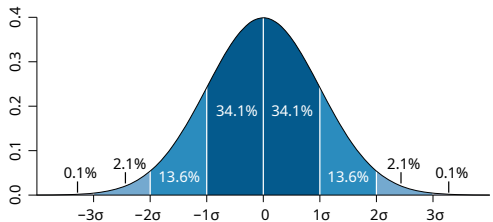
- Execute a sequence of M loads and stores of double precision floats N times
  (here $N = 10$, $M = 16$)
- Tested processor: TI OMAP4 with ARM Cortex-A9 cores



☞ Statistical distributions are likely **asymmetric** (e.g. no normal distributions)

Statistical Analysis

- Latency (or resource utilization times) are **statistical quantities**
  - Multiple measurements will result be different
- A statistical analysis can involve the following steps:
  - Estimation of a central value
  - Estimation of a distribution width
  - Estimation of the distribution (histogram)



Arithmetic Mean



- **Arithmetic mean** assuming N measurements $\Delta t^{(k)}$ have been performed:

$$\overline{\Delta t} = \frac{1}{N} \sum_{k=1}^{N} \Delta t^{(k)}$$

- **Challenge**
  - The average value is not a good estimator of a central value in case of asymmetric distributions
  - A single large of outlier can strongly affect the estimate (unless N is very large)
- **Note**: The distribution of latency measurements is typically not symmetric and large outliers may occur

Minimal Value

- **Minimal value** assuming N measurements $\Delta t^{(k)}$ have been performed:

$$\Delta t^{(\min)} = \min\left(\Delta t^{(k)}\right)$$

- The minimal value is robust with respect to outliers
- **Note**: Often there is a minimal latency but no maximum latency

Percentile (1/2)

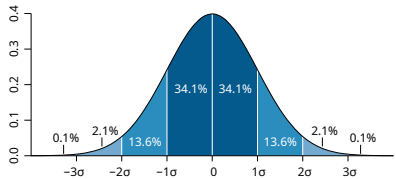
- **Percentile** = A number that denotes the position of a data point within a numeric dataset by indicating the percentage of the dataset with a lesser value

- Determination of a percentile Pk with $0 < k < 100$
  - Assume N measurements to be ordered such that

$$\Delta t^{(1)} \leq \Delta t^{(2)} \leq \cdots \leq \Delta t^{(N)}$$

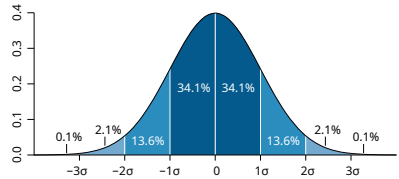
  - For sufficiently large N

$$Pk = \Delta t^{(\lceil k \cdot N/100 \rceil)}$$

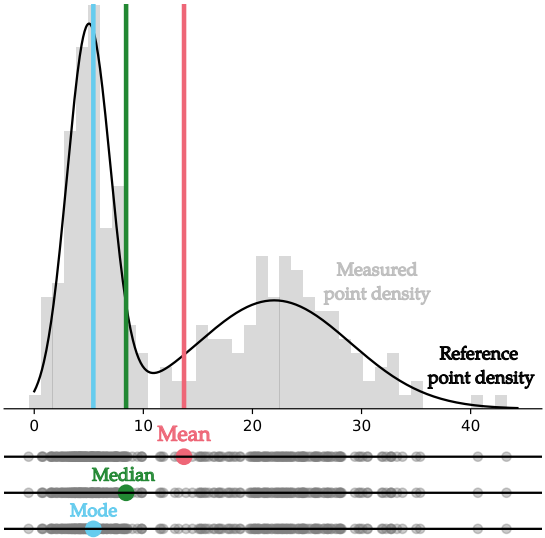


Percentile (2/2)

- Estimate of central value: $P50$
  - Also known as: **Median**, **second quartile**
- Variability estimate obtained by computing
  - First quartile $P25$
  - Third quartile $P75$
- Benefits of using the median:
  - Relatively easy to compute
  - Robust wrt outliers
  - Suitable for asymmetric probability distributions
  - Distribution width can be estimated using the P25 and P50 percentiles



Comparison



[Wikimedia; 2026]

Using Fits



- **Idea**: Perform measurements for different number of benchmark iterations
- **Assumption**:

$$\Delta t(N_{\text{iter}}) = N_{\text{iter}} \cdot \Delta t(1)$$

- **Approach**:
  - Measure $\Delta t(N_{\text{iter}})$ for different values of N_{iter}
  - Estimate benchmark execution time $\Delta \tilde{t}$ from fit to

$$\Delta t(N_{\text{iter}}) = c + N_{\text{iter}} \cdot \Delta \tilde{t}$$

Content



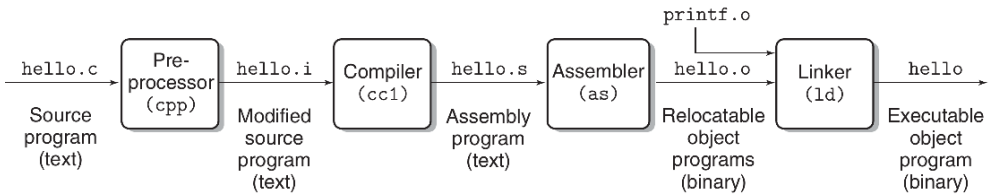
1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Compiler Chain: Overview



- Figure shows C compiler chain
- Other (imperative programming) languages similar

(C) Compiler Chain: Details



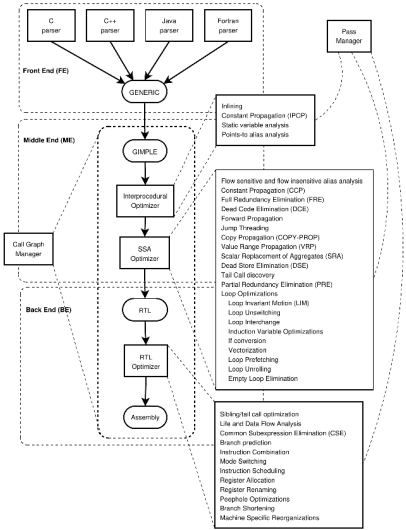
- **Pre-processing phase**: Source-to-source transformation
  - Can be exploited to turn run-time to compile-time computations
- **Compilation phase**: Translate high-level program to sequence of assembly instructions
  - Each assembly language statement corresponds to a machine instruction
  - Transformation from (largely) machine-agnostic to machine-specific code
- **Assembly phase**: Create relocatable object programs
  - Translates assembly into machine instructions
  - Typically, no further optimizations at this level
- **Linker phase**: Combine multiple relocatable object programs to an executable program

Main Compiler Tasks



- Code generation
- Instruction memory management (code blocks, address binding of instructions)
- Data memory management (stack variables)

GCC Organization (1/2)



[Diego Novillo; 2006]

GCC Organization (2/2)



- **Front-end**: Interpret different high-level languages and map to a generic representation
  - **Intermediate Representation** (IR) called GENERIC
- **Middle-end**:
  - Transformation to the GIMPLE IR
  - **Interprocedural optimizer**
  - **Static Single Assignment (SSA) optimizer**
- **Back-end**:
  - Transformation to the Register Transfer Language (RTL) IR
  - **RTL optimizer**
  - Assembly code generator

Compiler Optimization Steps



- **Legality-check**: Identify opportunities for code transformations and check correctness
  - A performance engineer can help by
    - Ensuring all necessary information being available
    - Making it easier for compilers to identify opportunities
- **Profitability-check**: Assess the benefits of the code transformation
  - Step based on heuristics that may be wrong
  - Sometimes a performance engineer can help by providing additional information
- **Transformation**: Implement the code transformation correctly

Selected Compiler Optimization Techniques (1/2)



- **Interprocedural optimizer**
  - **Function Inlining**
  - **Constant Propagation**
  - ...
- **Static Single Assignment (SSA) optimizer**
  - **Dead Code Elimination**
  - Loop Optimizations
    - Loop Interchange
    - **Loop Unrolling**
    - **Vectorization**
  - ...

Selected Compiler Optimization Techniques (2/2)



- **RTL optimizer**
  - **Common Subexpression Elimination**
  - Branch prediction
  - Instruction Combination
  - Register Allocation
  - Register Renaming
  - Peephole Optimizations
  - **Machine-Specific Reorganizations**
  - ...

Why is this Knowledge Relevant?



- Have an understanding of what compilers can and cannot do
- Understand what a compiler could have done
- Be able to check whether compilers decided to apply certain optimization techniques

GCC Optimization-Related Command-Line Options



- `O<#>`
  - `O0`: Disable optimizations
  - `O1 ... O3`: Increase optimization levels
- Different optimizations can also be enabled individually
- Some optimizations can be influenced using parameters
  - Usually not needed
- For others see, e.g.,
  `https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Optimize-Options.html`
- Note: Optimization can also be controlled using compiler-specific pragma statements

Optimization Caveats

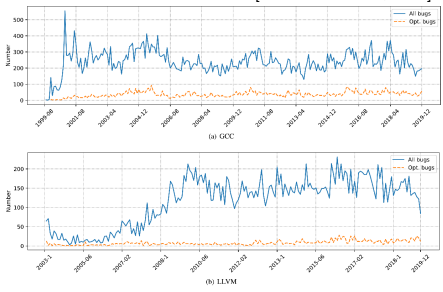


- Increased risk of bugs
- Change of rounding errors
- Code transformations could have a negative performance impact

Optimization Bugs

- Empirical research indicates that the optimizers are the most buggy part of GCC and LLVM
- This assessment is based on analysis of bug tickets
- Optimizations with the largest number of GCC bug tickets:
  - Value range propagation
  - Value numbering
  - Instruction combination
  - Partial redundancy elimination
- Note: The analysis does not take compiler release stage into account

[Zhide Zhou et al.; 2021]



Other Compilers



- LLVM
  - Most popular open-source compiler next to GCC
  - While GCC is GPL licensed, LLVM uses the Apache Licence
  - Recommended to test as an alternative to GCC
- Various vendor solutions
  - Often placed under a commercial licence (possible “community editions”)
  - Many are meanwhile based on LLVM
  - Goal: Provide machine-specific optimizations early
  - Examples:
    - Intel oneAPI compilers
    - AMD Optimizing C/C++ and Fortran Compilers (AOCC)
    - Arm Development Studio, Arm Compiler for Linux

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Assembler Generation



- Example GNU C-compiler: `gcc -S -O0 myprog.c`
  - Command will generate output file `myprog.s`
  - Usually better to start with zero-level optimization (`-O0`)
- Use cases:
  - Inspection of the instructions generated by the compiler
  - Starting point for own assembler implementations

Assembler Generation Example: x86



C program:

```c
void stupid_loop() {
  int i;
  for (i=0; i<100; i++);
}
```

Compilation:

```
gcc-14 -S -O0 myprog.c
```

Assembler:

```
        .file   "myprog.c"
        .text
        .globl  stupid_loop
        .type   stupid_loop, @function
stupid_loop:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    $0, -4(%rbp)
        jmp     .L2
.L3:
        addl    $1, -4(%rbp)
.L2:
        cmpl    $99, -4(%rbp)
        jle     .L3
        nop
        nop
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
```

x86 ISA



- ISA based on Intel 8086 CPU (launched 1978)
- Architecture dominates PC mass market and is used in most supercomputers
  - Top500: $93.4\,\%$ of the top500 systems use x86 CPUs [Top500, 06/2025]
- Manufacturers: Intel, AMD
- Main features
  - CISC-type ISA
  - One operand can be a memory location
- Extensions (selection):
  - x86-64
  - SSE2, SSE3, SSE4.x
  - AVX, AVX2, AVX-512

Arm ISA



- ARM = Advanced RISC Machine
  - Current main market: mobile and embedded electronics
  - Emerging server processor market
- ISA features
  - RISC-type ISA
  - Load/store architecture
- Extensions (selection):
  - SVE, SVE2
  - SME

Assembler Generation Example: Armv8



C program:

```
1 void stupid_loop() {
2   int i;
3   for (i=0; i<100; i++);
4 }
```

Compilation:

```
gcc-14 -S -O0 myprog.c
```

Assembler:

```
1           .arch armv8-a
2           .file   "myprog.c"
3           .text
4           .align  2
5           .global stupid_loop
6           .type   stupid_loop, %function
7   stupid_loop:
8   .LFB0:
9           .cfi_startproc
10          sub     sp, sp, #16
11          .cfi_def_cfa_offset 16
12          str     wzr, [sp, 12]
13          b       .L2
14  .L3:
15          ldr     w0, [sp, 12]
16          add     w0, w0, 1
17          str     w0, [sp, 12]
18  .L2:
19          ldr     w0, [sp, 12]
20          cmp     w0, 99
21          ble     .L3
22          nop
23          nop
24          add     sp, sp, 16
25          .cfi_def_cfa_offset 0
26          ret
27          .cfi_endproc
```

Hints



- Start by compiling with -O0 to avoid difficult to understand Optimizations
- Use inline assembler statements to insert markers

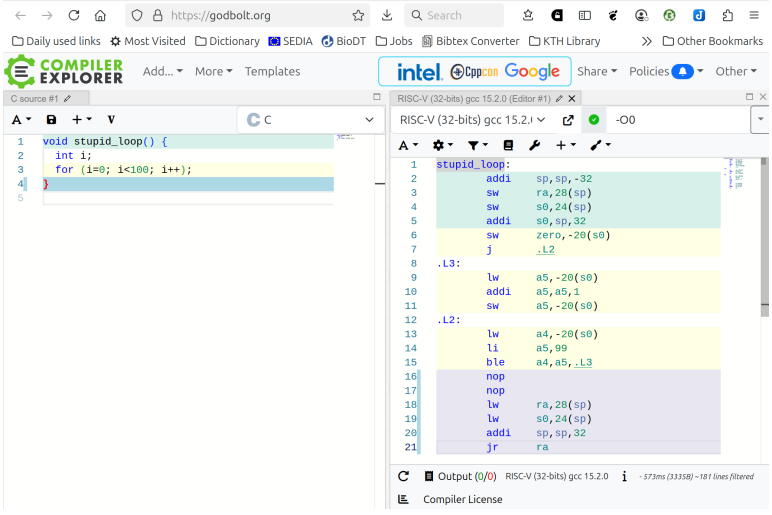
C program:

```
1 double f(double x, int n) {
2     asm("mymark1:");
3     for (int i = 0; i < n; i++)
4         x *= x;
5     asm("mymark2:");
6     return x;
7 }
```

Assembler:

```
16 #APP
17 # 2 "ex.c" 1
18         mymark1:
19 # 0 "" 2
20 #NO_APP
21         movl    $0, -4(%rbp)
22         jmp     .L2
23 .L3:
24         movsd   -24(%rbp), %xmm0
25         mulsd   %xmm0, %xmm0
26         movsd   %xmm0, -24(%rbp)
27         addl    $1, -4(%rbp)
28 .L2:
29         movl    -4(%rbp), %eax
30         cmpl    -28(%rbp), %eax
31         jl      .L3
32 #APP
33 # 5 "ex.c" 1
34         mymark2:
35 # 0 "" 2
36 #NO_APP
```

Godbolt


https://godbolt.org
Search
Daily used links
Most Visited
Dictionary
SEDIA
BioDT
Jobs
Bibtex Converter
KTH Library
Other Bookmarks
COMPILER EXPLORER
Add...
More
Templates
intel
Cppcon
Google
Share
Policies
Other
C source #1
C
RISC-V (32-bits) gcc 15.2.0 (Editor #1)
RISC-V (32-bits) gcc 15.2.0
-O0
void stupid_loop() {
int i;
for (i=0; i<100; i++);
}
stupid_loop:
addi sp,sp,-32
sw ra,28(sp)
sw s0,24(sp)
addi s0,sp,32
sw zero,-20(s0)
j .L2
.L3:
lw a5,-20(s0)
addi a5,a5,1
sw a5,-20(s0)
.L2:
lw a4,-20(s0)
li a5,99
ble a4,a5,.L3
nop
nop
lw ra,28(sp)
lw s0,24(sp)
addi sp,sp,32
jr ra
Output (0/0)
RISC-V (32-bits) gcc 15.2.0
- 573ms (33358) ~181 lines filtered
Compiler License

Content



1. Performance Measurement

2. Compiler Chain

3. Assembler

4. Final

Summary Lecture #02



- Performance measurement
  - Different performance measurement methods
  - Means for measuring performance and performance relevant observables
- Optimizations leveraging the compiler
- Analysis of assembler to understand code optimization or guide further code optimizations

Questions?



[Wiki Media]

These slides contain content and figures from

- R. E. Bryant and D. R. O'Hallaron, "Computer Systems: A Programmer's Perspective," 2nd edition, Pearson, 2011